Chess Position Recognition

Square-level board understanding from schematic chess images

Dataset	Kaggle Chess Positions
Model	Scratch residual CNN in PyTorch
Primary metric	Square accuracy over 64 board cells

Oğuzhan Özkaya

ADA 447 Introduction to Deep Learning

Project Goal

Project Goal

- Input: one aligned 400 by 400 RGB chessboard image.
- Output: one label for every board square.
- Label set: `empty`, white pieces, and black pieces.
- Success is measured primarily by square-level accuracy.

Prediction Target
64
squares per image

Classes
13
empty + 12 pieces

Main Metric
Accuracy
square-level correctness

Dataset

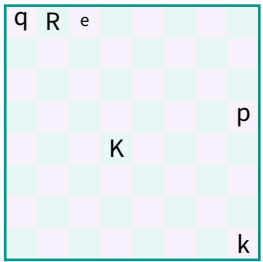
- Source: Chess Positions dataset.
- 100,000 generated schematic board images.
- 80,000 training images and 20,000 test images.
- Each image contains 5–15 pieces, including both kings.
- Images span many board and piece style combinations.
- Filename stems encode labels as FEN ranks with dashes instead of slashes.

Data Contract

Field	Requirement
Location	`data/train/` and `data/test/`
Image	RGB board image, read from disk every run
Label	Filename stem with 8 dash-separated FEN ranks
Split	Random validation split from `data/train/`; final test uses `data/test/`
Cache policy	No processed dataset cache is written

Label Parsing

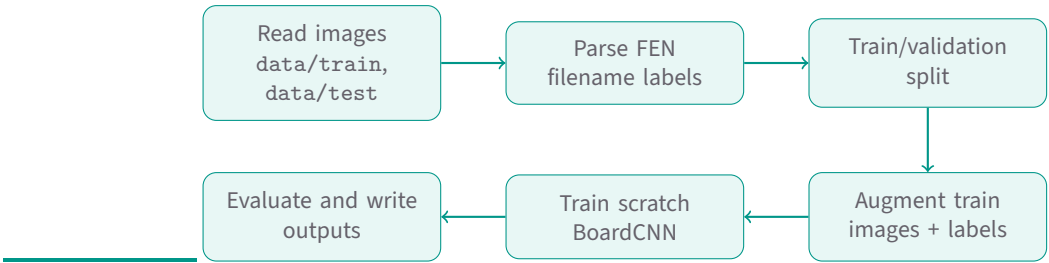
- A filename stem is split into 8 ranks.
- Digits expand to repeated `empty` squares.
- Piece letters map directly to class ids.
- The result is an `8 x 8` label grid.



`qR6-N7-...` becomes square labels.

Pipeline

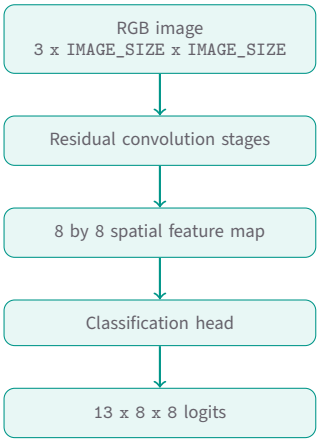
Pipeline



Model Architecture

Model Architecture

- Implemented directly in PyTorch as `BoardCNN`.
- Residual convolution blocks learn piece and board features.
- Spatial downsampling aligns features to an 8 by 8 grid.
- A 1 by 1 convolution emits class logits for each square.



Training Setup

Component	Setting
Optimizer	AdamW with weight decay
Loss	Cross-entropy over square labels
Empty handling	Lower empty-class weight to focus on occupied squares
Regularization	Dropout, label smoothing, and label-preserving augmentation
Hardware target	Local RTX 4080 with CUDA mixed precision
Checkpoint rule	Best model selected by validation square accuracy

Metrics

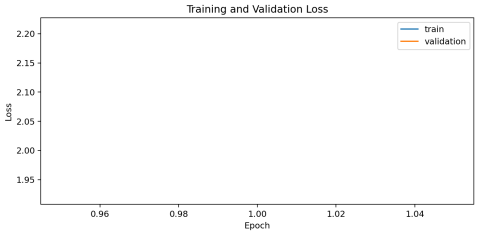
- **Square accuracy**: primary metric over every board square.
- **Occupied-square accuracy**: piece recognition without empty squares.
- **Empty-square accuracy**: empty-cell recognition quality.
- **Board accuracy**: strict metric requiring all 64 squares correct.
- **Per-class precision/recall/F1**: piece-specific behavior.

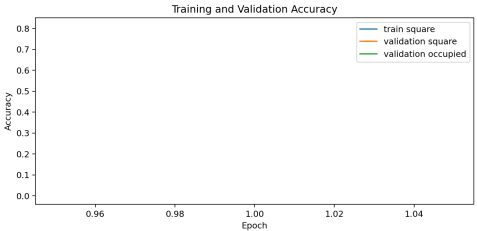
Generated Outputs

Path	Purpose
`output/models/`	PyTorch checkpoints and final model
`output/predictions/`	Square-level predictions in CSV and Parquet
`output/reports/`	Metrics, class reports, and training history
`output/figures/`	Training curves, confusion matrix, confidence plots

Training Curves

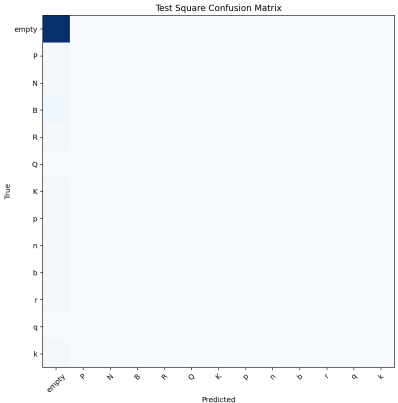
Training Curves

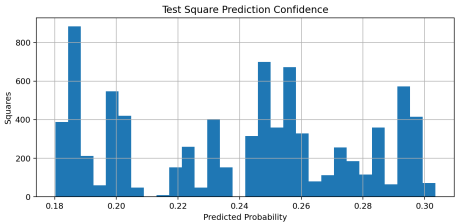




Evaluation Figures

Evaluation Figures





Results Placeholder

Results Placeholder

Split	Square Accuracy	Occupied Accuracy	Board Accuracy
Validation	From `metrics.csv`	After training	After training
Test	From `metrics.csv`	After training	After training

Rebuild this deck after the long run with `just presentation-build`.

Risks and Mitigations

Risk	Mitigation
Empty-square dominance	Down-weight empty class and track occupied-square accuracy
Style variation	Train on all board and piece styles with color augmentation
Overfitting	Validation split, dropout, label smoothing, early stopping
Slow full training	Mixed precision and RTX 4080 local execution
Report drift	Presentation references generated outputs directly

Reproducibility

- Install environment: `just sync`
- Smoke test: `just smoke`
- Full training: `just run`
- Build docs: `just docs-build`
- Build presentation: `just presentation-build`

The project reads raw images on each run and writes all report artifacts under `output/`.

Conclusion

- The project has been migrated to chess board position recognition.
- The model predicts all 64 square labels with one scratch CNN.
- Accuracy is measured at square, occupied-square, empty-square, and full-board levels.
- The remaining project work is the long RTX 4080 training run and final result interpretation.

Questions

Chess Position Recognition